Parking Lot System

Problem Statement

Background

A company runs a multi-floor parking lot for cars, bikes and trucks. You are required to design and build a system that manages the parking lot setup, vehicle entry and slot allocation, tickets and exit, fee calculation and payments, and monitoring reports. The system must follow the business rules described in each module below.

Module 1 — Parking Lot Setup

- Only an Admin can set up the parking lot, its floors and its slots.
- A parking lot has a name, an address and at least one floor. Each floor has at least one slot.
- A slot has a number (unique on its floor), a type (Bike, Car, or Truck) and a status (Free, Occupied, or Under Maintenance).
- A slot's type cannot be changed while it is Occupied. Only a Free slot can be marked Under Maintenance.
- An Admin can close a floor. No new vehicles are allotted to a closed floor, but vehicles already parked there can exit normally.

Module 2 — Vehicle Entry and Slot Allocation

- A vehicle has a unique registration number, a type (Bike, Car, or Truck) and an owner name.
- A vehicle that is already parked (has an active ticket) cannot enter again.
- A Bike gets a Bike slot, a Car gets a Car slot and a Truck gets a Truck slot. If no Bike slot is free, a Bike can use a Car slot. A Car cannot use a Bike slot, and a Truck can use only a Truck slot.
- The system allots the nearest free slot: the lowest floor number first, then the lowest slot number.
- If no suitable slot is free, the entry is rejected with a Parking Full message.

Module 3 — Ticket and Exit

- A ticket is generated on entry with a unique ticket ID, the registration number, the vehicle type, the slot and the entry time.
- Ticket status is Active, Paid, Closed or Lost.
- A vehicle can exit only when its ticket is Paid, and only within 15 minutes of the payment. After that, the extra time is charged again.
- On exit, the slot becomes Free and the ticket becomes Closed.
- If a ticket is lost, the owner must give the registration number and pay a flat penalty of Rs. 500 in addition to the parking fee.
- A vehicle parked for more than 24 hours continuously is flagged to the Admin.

Module 4 — Fee Calculation and Payment

- Hourly rate: Bike Rs. 10, Car Rs. 20, Truck Rs. 40. The first 15 minutes are free. After that, the time is rounded up to the next full hour.
- The fee for one day is capped at 8 hours of the hourly rate.
- A Bike parked in a Car slot is charged at the Bike rate.
- A vehicle with a valid monthly pass is not charged. A pass has the registration number, a start date and an end date. Monthly pass fee: Bike Rs. 500, Car Rs. 1,500, Truck Rs. 3,000. An expired pass is treated as no pass.
- Payment methods are Cash, UPI and Card. A payment record must store the amount, date, method and status (Paid or Failed). If the payment fails, the vehicle cannot exit.
- A receipt is generated for every payment.

Module 5 — Reports and Monitoring

- An Admin can view the number of free and occupied slots by floor and by slot type.
- An Admin can view the total revenue for a given day and the list of vehicles currently parked.
- When the occupancy of a floor goes above 90%, an alert is shown to the Admin.

- A slot Under Maintenance is not counted as free or occupied in the reports.

SQL Exercise

Based on the system described above, write SQL queries for the following:

Query 1

For each vehicle type, find the total number of tickets issued, the total revenue collected, and the average parking duration in minutes. Show only vehicle types that have more than 2 tickets. Sort by total revenue in descending order.

Query 2

For each floor, list the total number of slots, the number of tickets issued, and the total revenue collected. Group the results by floor and include only floors that have issued at least 3 tickets.

Query 3

Find each vehicle's total number of visits, the total fee paid, and the number of distinct floors it has parked on. Include only vehicles that have visited more than 2 times, ordered by total fee paid in descending order.

Instructions

Diagrams

Draw an Entity Relationship diagram showing all the database tables you would create and the relationships between them (one-to-many, many-to-many, etc.). Also draw a high-level system diagram showing how the modules interact with each other.

Application

Build a console application. You do not need to connect to any database.

Data Storage

Use in-memory collections (such as HashMap, List, or equivalent in your language) to store and retrieve all records at runtime. You can hardcode a few parking floors, slots of each type and a few vehicles with monthly passes as initial data to start the program. All other data — tickets, payments, receipts, passes, alerts — must be handled dynamically: add to the collection when created, update in place, and remove when deleted or cancelled.

OOP Concepts

Your code must clearly demonstrate the following object-oriented concepts:

- **Encapsulation** — keep fields private and expose them through methods.
- **Inheritance** — identify entities that share common properties and model them using a base class (for example, Bike, Car and Truck can extend a Vehicle base class, and Bike, Car and Truck slots can extend a Slot base class).
- **Polymorphism** — handle different behaviors through a common type (for example, each vehicle type calculating its own parking fee).
- **Abstraction** — hide details behind an abstract class or interface (for example, a FeeCalculator interface or a PaymentMethod interface).